UNIT - I : PYTHON BASICS AND CONTROL FLOW

1. History of Python

* Created by GUIDO VAN ROSSUM at CWI (Netherlands) in the late 1980s (started Dec 1989), first released Feb 1991 (version 0.9.0). Named after the comedy show 'Monty Python's Flying Circus', NOT the snake.
* Python 1.0 - 1994. Python 2.0 - 2000 (list comprehension, garbage collection). Python 3.0 - December 2008 (not backward compatible). Python 2 reached end of life on 1 January 2020. Managed by the Python Software Foundation (PSF).
* Influenced by ABC, C, Modula-3, Lisp, Haskell, Java. Features: simple and readable syntax, interpreted, high-level, dynamically typed, object oriented, free and open source, portable (platform independent), extensible, huge standard library ('batteries included'), automatic memory management.

Thrust areas of Python

* Web development (Django, Flask), Data science and analytics (NumPy, Pandas, Matplotlib), Machine learning / AI (scikit-learn, TensorFlow, PyTorch), Scientific computing (SciPy), Automation and scripting, Game development (Pygame), Desktop GUI (Tkinter), Networking / cyber security, Embedded and IoT (Raspberry Pi, MicroPython), Web scraping, Database access, Education.

2. Installing Anaconda and Jupyter Notebook

* ANACONDA = free distribution of Python and R for data science. It includes Python, 250+ packages (NumPy, Pandas, Matplotlib, SciPy), the conda package manager, Jupyter Notebook, Spyder IDE. Available for Windows, macOS, Linux.
* Steps: (1) Go to anaconda.com and download the installer for your OS

(64-bit, Python 3.x). (2) Run the installer, accept the licence, choose 'Just Me', select install location. (3) Finish; open 'Anaconda Navigator' or 'Anaconda Prompt'. (4) Check with:

```
> conda --version
> python --version
> conda install numpy        # install a package
> conda list                 # show installed packages
```

Jupyter Notebook

* Web based interactive environment: a notebook (.ipynb) holds CODE cells, MARKDOWN cells, output, graphs. Launch from Anaconda Navigator -> Launch Jupyter Notebook, or type 'jupyter notebook' at the prompt; opens in browser at localhost:8888.
* Use: New -> Python 3 notebook; type code in a cell; run with SHIFT + ENTER (CTRL + ENTER runs without moving); A / B insert cell above / below; D D delete cell; M markdown; Y code. Save as .ipynb; File -> Download as .py.

3. Parts of Python Programming Language

Identifiers

* Names given to variables, functions, classes, modules. Rules: start with a letter (A-Z, a-z) or underscore; followed by letters, digits, underscores; CASE SENSITIVE (age and Age are different); cannot be a keyword; no spaces or special symbols (@, $, %).

```
valid:   name, _count, total2, Student_Name
invalid: 2cost, my-name, class, for, a b
```

Keywords

* Reserved words with special meaning; cannot be used as identifiers. Python

3 has 35 keywords (changes with version): False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield. Only True, False, None start with capital.

```
import keyword
print(keyword.kwlist)
print(len(keyword.kwlist))
```

Statements and Expressions

* STATEMENT = an instruction that the interpreter executes (assignment, if, for, print, import). EXPRESSION = a combination of values, variables and operators that evaluates to a single VALUE (3 + 4 * 2). Every expression is part of a statement.
* A statement normally ends at the end of the line. Use backslash (\) or brackets for a multi-line statement; use semicolon (;) to put more than one statement on a line.

```
x = 10 + 5              # assignment statement, expr 10+5
total = 1 + 2 + \
        3 + 4           # line continuation
a = 1; b = 2            # two statements in one line
```

Variables

* A variable is a NAME that refers to an object (value) stored in memory. No declaration needed; type is decided by the value assigned (dynamic typing). Assignment operator is '='.

```
x = 5              # int
name = "Ravi"      # str
a = b = c = 0      # multiple assignment
p, q, r = 1, 2.5, "hi"   # multiple values
print(id(x))       # memory address of object
```

Operators

Type	Operators	Example
Arithmetic	+ - * / // % **	7//2 = 3, 7%2 = 1, 2**3 = 8, 7/2 = 3.5
Relational (comparison)	== != > < >= <=	5 > 3 is True
Assignment	= += -= *= /= //= %= **= &= |= ^= >>= <<=	x += 2 means x = x + 2
Logical	and or not	(5>3) and (2>1) is True
Bitwise	& | ^ ~ << >>	5 & 3 = 1, 5 | 3 = 7, 5 ^ 3 = 6, ~5 = -6, 5<<1 = 10
Membership	in not in	'a' in 'cat' is True
Identity	is is not	x is y (same object?)
Ternary (conditional)	value1 if condition else value2	big = a if a > b else b

Precedence and associativity

* PRECEDENCE decides which operator is evaluated first; ASSOCIATIVITY decides the order when operators have same precedence (mostly LEFT to right; ** and assignment are RIGHT to left).

Order (high -> low)	Operator
1	( ) parentheses
2	** exponent (right to left)
3	+x -x ~x unary
4	* / // %
5	+ -
6	<< >>
7	& then ^ then |
8	== != > < >= <= is is not in not in
9	not then and then or
10	if - else (ternary), = (assignment)

```
>>> 2 + 3 * 4          # * first
14
>>> 2 ** 3 ** 2        # right to left: 2 ** 9
512
>>> 100 / 10 * 2       # left to right
20.0
>>> not 1 > 2 and 3 < 5
True
```

Data types

* Everything is an object. Built-in types: NUMERIC: int (unlimited size), float (64-bit, 15 digits), complex (3+4j); BOOLEAN: bool (True / False); TEXT: str; SEQUENCE: list, tuple, range; MAPPING: dict; SET: set, frozenset; BINARY: bytes, bytearray; NONE: NoneType (None).

Type	Example	Mutable?
int	10, -45, 0b101, 0o17, 0xFF	No
float	3.14, 2.5e3	No
complex	2+3j (z.real, z.imag)	No
bool	True, False (= 1, 0)	No
str	'hello', "hi", '''multi'''	No
list	[1, 'a', 2.5]	Yes
tuple	(1, 2, 3)	No
set	{1, 2, 3}	Yes
dict	{'a': 1, 'b': 2}	Yes

Indentation

* Python uses INDENTATION (whitespace at start of a line) to define blocks - there are no braces { }. Standard is 4 spaces. All statements in a block must be indented equally; wrong indentation gives IndentationError. Block header ends with a colon ':'.

```
if 5 > 2:
    print("five is greater")    # inside block
    print("done")
print("outside block")
```

Comments

* Single line comment starts with # (ignored by the interpreter). Multi-line comment: use several # lines or triple quoted string (''' ... ''' - actually a docstring when first in a function / class).

```
# this is a comment
x = 5   # inline comment
'''
multi line
comment
'''
```

Reading input and printing output

* input(prompt) reads a line from keyboard and ALWAYS returns a STRING - convert with int(), float(). print() displays output; parameters sep (separator, default space), end (default newline), file.

```
name = input("Enter name: ")
age = int(input("Enter age: "))
print("Hello", name, "you are", age)
print("a", "b", "c", sep="-")        # a-b-c
print("Hi", end=" ")                 # no newline
print("there")                       # Hi there
print(f"{name} is {age} years old")  # f-string
print("%s is %d" % (name, age))      # % formatting
print("{} is {}".format(name, age))  # format()
```

Type conversions, type() and is

* IMPLICIT conversion: Python converts automatically (int + float -> float). EXPLICIT (type casting): int(), float(), str(), bool(), list(), tuple(), set(), complex(), chr(), ord().

```
>>> int("25") + 5
30
>>> float(7)
7.0
>>> int(3.99)            # truncates
3
>>> str(10) + "5"
'105'
>>> bool(0), bool("a")
(False, True)
>>> 5 + 2.0              # implicit -> float
7.0
>>> type(5), type(5.0), type("5"), type([1])
(<class 'int'>, <class 'float'>, <class 'str'>, <class 'list'>)
>>> a = [1]; b = a; c = [1]
>>> a is b, a is c, a == c
(True, False, True)
```

'==' compares VALUES; 'is' compares IDENTITY (same object in memory, id(a) == id(b)).

Dynamic and strongly typed language

* DYNAMICALLY typed: type of a variable is checked at RUN time and the same name can refer to different types: x = 5 then x = 'five' is allowed (no type declaration).
* STRONGLY typed: Python does NOT implicitly mix incompatible types - '5' + 3 raises TypeError; you must convert explicitly: int('5') + 3 = 8. (Weakly typed languages like JavaScript would give '53').

```
x = 10
print(type(x))       # <class 'int'>
x = "ten"
print(type(x))       # <class 'str'>
print("5" + 3)       # TypeError: can only concatenate str
```

4. Control Flow Statements

if statement

```
age = 20
if age >= 18:
    print("Eligible to vote")
```

if - else

```
n = int(input("Number: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

if ... elif ... else (ladder)

```
marks = 72
if marks >= 90:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Fail"
print(grade)            # B
```

Nested if

```
a, b, c = 5, 9, 7
if a > b:
    if a > c:
        print("a is largest")
    else:
        print("c is largest")
else:
    if b > c:
        print("b is largest")
    else:
        print("c is largest")
```

while loop

* Repeats a block as long as the condition is True. Optional else: runs when loop ends normally (not by break). Beware of infinite loops.

```
i = 1
while i <= 5:
    print(i, end=" ")
    i += 1              # 1 2 3 4 5
else:
    print("loop finished")
```

for loop and range()

* for iterates over a sequence (list, string, tuple, dict, range). range(stop), range(start, stop), range(start, stop, step) gives numbers from start up to stop-1.

```
for i in range(5):              # 0 1 2 3 4
    print(i, end=" ")
for i in range(2, 10, 3):       # 2 5 8
    print(i, end=" ")
for ch in "abc":
    print(ch)
for i in range(10, 0, -2):      # 10 8 6 4 2
    print(i, end=" ")
```

break, continue and pass

* break - terminates the loop immediately. continue - skips the rest of the current iteration and goes to the next. pass - null statement, does nothing (placeholder).

```
for i in range(1, 10):
    if i == 3:
        continue       # skip 3
    if i == 6:
        break          # stop at 6
    print(i, end=" ")  # 1 2 4 5
```

Catching exceptions - try and except

* An EXCEPTION is a run-time error (ZeroDivisionError, ValueError, TypeError, IndexError, KeyError, FileNotFoundError, NameError). try block holds risky code; except handles it, so the program does not crash. else runs if no exception; finally ALWAYS runs (cleanup). raise throws an exception.

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Enter integers only")
except Exception as e:
    print("Error:", e)
else:
    print("No error")
finally:
    print("Done")
```

5. Sample Experiments (1 - 6)

Exp 1: Largest of three numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a >= b and a >= c:
    large = a
elif b >= a and b >= c:
    large = b
else:
    large = c
print("Largest number is", large)
# Output: inputs 12 45 30 -> Largest number is 45
```

Exp 2: Prime numbers within an interval

```
lo = int(input("Lower limit: "))
hi = int(input("Upper limit: "))
print("Primes between", lo, "and", hi)
for num in range(lo, hi + 1):
    if num > 1:
        for i in range(2, int(num ** 0.5) + 1):
            if num % i == 0:
                break
        else:                   # no divisor found
            print(num, end=" ")
# Output (10 to 30): 11 13 17 19 23 29
```

Exp 3: Swap two numbers without temporary variable

```
a = int(input("a = "))
b = int(input("b = "))
print("Before swap:", a, b)
a = a + b
b = a - b
a = a - b
print("After swap:", a, b)
# other ways:  a, b = b, a      or   a = a ^ b; b = a ^ b; a = a ^ b
# Output: a=5, b=9 -> After swap: 9 5
```

Exp 4: Demonstration of operators

```
a, b = 10, 3
print("Arithmetic:", a+b, a-b, a*b, a/b, a//b, a%b, a**b)
print("Relational:", a>b, a<b, a==b, a!=b, a>=b, a<=b)
x = 10
x += 5; print("Assignment +=:", x)     # 15
x *= 2; print("Assignment *=:", x)     # 30
print("Logical:", a>5 and b<5, a>5 or b>5, not(a>5))
print("Bitwise:", a&b, a|b, a^b, ~a, a<<1, a>>1)
big = a if a > b else b                # ternary
print("Ternary:", big)
l = [1, 2, 3]
print("Membership:", 2 in l, 5 not in l)
p = [1, 2, 3]; q = l; r = [1, 2, 3]
print("Identity:", l is q, l is r, l is not r)
# Output
# Arithmetic: 13 7 30 3.33.. 3 1 1000
# Bitwise: 2 11 9 -11 20 5
```

Exp 5: Add and multiply complex numbers

```
c1 = complex(2, 3)              # 2+3j
c2 = complex(4, -1)             # 4-1j
print("Sum     =", c1 + c2)     # (6+2j)
print("Product =", c1 * c2)     # (11+10j)
# (2+3j)(4-j) = 8 -2j +12j -3j^2 = 11 + 10j
print("Real part:", c1.real, " Imag:", c1.imag)
```

Exp 6: Multiplication table of a number

```
n = int(input("Enter a number: "))
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
# Output (n = 7):  7 x 1 = 7 ... 7 x 10 = 70
```

Revision: Python 3.x - created by Guido van Rossum (1991). input() returns str. 'is' = identity, '==' = equality. Strongly + dynamically typed. Indentation = blocks. try-except-else-finally for exceptions.